GREEN UNIVERSITY OF BANGLADESH

Industrial Training

on

Full-Stack Web Development

Submitted by

Jubaer Al Mamun – 221902013

An industrial training report submitted to the Department of Computer Science & Engineering, for the partial fulfillment of the degree of Bachelor of Science in Computer Science & Engineering

Under the Supervision of

Md. Parvez Hossain

Lecturer, Department of CSE

Training Place

Tanbel Inc. (Parent of WeCycle)

United States (Remote)

Training Duration :	13.10.2025 to 28.11.2025
Industrial Supervisor :	MD Tanvir Ahmed
Report Submission Date :	26.12.2025

Fall 2025

Declaration

Declaring this training report to be founded only on my own research results, I make it clear. References are made to research materials discovered by other researchers. This work has not been submitted for a degree before, either in full or in part.

Jubaer Al Mamun

ID: 221902013

Certificate

This is to certify that the industrial training report entitled *Full-Stack Web Development* has been prepared and submitted by **Jubaer Al Mamun** in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering on 26 December 2025.

Md. Parvez Hossain	MD Tanvir Ahmed
Academic Supervisor	Industrial Supervisor
Green University of Bangladesh	Tanbel Inc.

Accepted and approved in partial fulfillment of the requirements for the degree Bachelor of Science in Computer Science and Engineering.

Chairman

Board Member	Board Member

Acknowledgments

I would like to express my sincere gratitude to all those who supported and guided me throughout the industrial training period from **October 13, 2025** until **December 5, 2025**. Their encouragement and assistance were invaluable, especially during times when I faced challenges.

I am thankful to my academic supervisor, **Md. Parvez Hossain, Lecturer, Department of CSE, Green University of Bangladesh**, for providing consistent guidance and support during the training. Their feedback and suggestions helped me stay focused and improved the overall quality of my learning experience.

I am also grateful to the professionals at **Tanbel Inc.** (specifically the **WeCycle** team), especially my industrial supervisor **MD Tanvir Ahmed**, for their valuable insights, technical guidance, and for sharing their industry experience. Their support greatly enhanced my understanding of real-world practices in **Full-Stack Web Development**, modern JavaScript frameworks, and agile work culture.

I would also like to acknowledge the entire development team at WeCycle for their patience in reviewing my code, providing constructive feedback, and helping me understand the complexities of working with large-scale web applications.

Finally, I would like to thank my family and friends for their constant encouragement, emotional support, and motivation, which played a vital role in helping me successfully complete the training.

Abstract

This report summarizes the industrial training experience completed at **Tanbel Inc.**, a US-based technology company. The training utilized their two primary products: **WeCycle** (https://getwecycle.com/), a B2C junk removal service, and **ArrayQ** (https://arrayq.com/), a B2B hauler dashboard CRM solution. The training focused on the practical application of **Full-Stack Web Development** and mobile application maintenance. The primary objective was to gain hands-on experience in developing scalable user interfaces and backend integrations for large enterprise applications.

The training involved working with a modern technology stack, including **React.js, Next.js, TypeScript, Tailwind CSS, Capacitor, and Docker**. Industry-standard tools like **Jira** for project management and **GitHub** for version control were integral to the daily workflow.

Key contributions included **developing the ArrayQ landing page and mobile app features, implementing a complex invoice and payment system using Stripe, integrating an AI-powered chatbot for customer support, and migrating legacy Ant Design components to a custom UI library**. Extensive work was also done on optimizing the **WeCycle Dashboard** for iPad and tablet devices, fixing critical bugs, and setting up staging environments for rigorous testing.

This internship provided a comprehensive learning experience, bridging the gap between academic theory and professional software engineering. It offered deep insights into agile development cycles, code quality standards, and the operational dynamics of a remote tech startup, preparing me for a career as a **Full-Stack Developer**.

Keywords: Tanbel Inc., WeCycle, ArrayQ, React.js, Next.js, TypeScript, Capacitor,

Stripe Integration, AI Chatbot

TABLE OF CONTENTS

Declaration i

Certificate ii

Acknowledgments iii

Abstract iv

List of Figures x

List of Tables xi

1 Introduction 1

1.1 Problem Domain 1

1.2 Motivation of Training 2

1.3 Training Goals 3

1.4 Scope of Training 3

1.5 Scope of Training 3

1.6 The Reality Check: Academia vs. Industry 4

1.7 Pain Points & Growth 5

1.7.1 The Hard Stuff (Challenges) 5

1.7.2 Where I Grew 5

1.8 Detailed Plan 5

1.8.1 Resources 5

1.8.2 Timeline 6

1.9 Risk Analysis: What Could Go Wrong? 6

1.10 Report Outline 7

1.11 Conclusion 7

2 Organization Overview 8
2.1 Introduction 8
2.2 Company Profile 8
2.2.1 Company Background 9
2.2.2 Key Services 9
2.2.3 Work Environment 10
2.2.4 Mission and Vision 10
2.2.5 Core Values 10
2.3 Organizational Structure 11
2.3.1 Type of Organization 11
2.3.2 Hierarchical Overview 11
2.3.3 Interactions with Various Departments 12
2.3.4 Collaboration with Team Leads 12
2.3.5 Cross-Functional Coordination 12
2.4 Products/Services 13
2.4.1 WeCycle Dashboard Application (WCAPP) 13
2.4.2 Technology Stack 17
2.5 Clients 17
2.6 Direct Involvement of Resources During Training 17
2.7 Constructive Feedback About the Training Program 18
2.8 Conclusion 19

3 Specific Details on Training Activities and Report 20
3.1 Introduction 20
3.2 Weekly Duties and Activities 21
3.3 Weekly Duties and Activities 21
3.3.1 Week 1: Bootstrapping & Environment Setup (October 13-17, 2025) 21
3.3.2 Week 2: Infrastructure & Deployment (October 20-24, 2025) 21

3.3.3 Week 3: Data Management & Optimization (October 27-31, 2025) . . . 22
3.3.4 Week 4: Branding & ArrayQ Development (November 03-07, 2025) . . . 22
3.3.5 Week 5: Deployment & Role Management (November 10-14, 2025) . . . 23
3.3.6 Week 6: UX Refinements & Performance (November 17-21, 2025) . . . 23
3.3.7 Week 7: Pricing & Pre-Production (November 24-28, 2025) . . . 24
3.3.8 Week 8: Fintech & Final Polish (December 01-05, 2025) . . . 24
3.4 Key Projects . . . 25
3.4.1 Project 1: ArrayQ Platform Development . . . 25
3.4.2 Project 2: Invoice & Payment System Modernization 25
3.4.3 Project 3: Chatbot & Support Integration . . . 26
3.5 Other Responsibilities . . . 27
3.6 Internship Summary . . . 27

4 Acquired Skills During Industrial Training . . . 30
4.1 Introduction . . . 30
4.2 Tech Stack: What I Actually Learned . . . 30
4.2.1 React.js and Next.js [1, 2] . . . 30
4.2.2 Frontend: React.js & Next.js . . . 31
4.2.3 Backend: Node.js & Databases . . . 31
4.2.4 Mobile: Capacitor & React Native . . . 32
4.2.5 DevOps: Docker & CI/CD . . . 32
4.2.6 TypeScript . . . 32
4.2.7 CSS/SCSS, Tailwind CSS, and Responsive Design [3] 32
4.2.8 Git and Version Control . . . 33

4.3 Theoretical Knowledge vs. Reality 33
4.3.1 Software Development Concepts 33
4.3.2 Development Methodologies 34
4.4 Practical Knowledge: The ”How-To” 34
4.5 Soft Skills (The Stuff That Actually Matters) 35
4.6 Collaborating Without Killing Each Other 35
4.7 Ethics & Professionalism 35
4.8 Conclusion 36

5 Conclusion 37
5.1 Summary 37
5.2 The Real Challenges 38
5.3 What I Actually Did (Scope) 38
5.4 Problems I Solved 39
5.5 The Verdict (Overall Achievement) 39

Annexure: PO-Skill Mapping 41

Annexure: SDG–Goal Mapping 45

Appendix 50

References 53

List of Symbols 55

List of Figures

2.1 Tanbel Inc. (WeCycle) Organizational Structure 12
2.2 WeCycle Dashboard - Job Scheduling and Management 14
2.3 WeCycle Dashboard - Client Management 14
2.4 WeCycle Dashboard - Crew Management 15
2.5 WeCycle Dashboard - Invoice Management 15
2.6 WeCycle Dashboard - Team Chat 16
2.7 WeCycle Dashboard - Calendar View 16

3.1 Agile Scrum Workflow Used During Industrial Training 28

List of Tables

1.1 Project Timeline Planning - Weekly Task Distribution 6

2.1 What Tanbel Inc. (WeCycle ArrayQ) Actually Does 9

2.2 Technology Stack Configuration 17

3.1 Internship Summary 29

4.1 How I Kept It Professional 36

5.1 Program Outcome (PO) Fulfillment in Industrial Training 42

5.2 Mapping of All 17 SDG Goals with Achievements from Industrial Training 46

5.3 Weekly Engineering Log: Key Milestones 50

Chapter 1

Introduction

The transition from academic theory to professional engineering is often abrupt. While university courses provide a strong foundation in algorithms and syntax, they rarely simulate the volatility of a production environment. In a classroom, code is written to pass test cases; in the industry, code is written to survive user interaction, scalability demands, and market pivots. This industrial training at **Tanbel Inc.** was not just an application of skills—it was an immersion into the reality of modern software development. I moved from writing isolated functions to contributing to a living, breathing product ecosystem, where ”urgent” means ”now,” and stability is the primary metric of success.

The objective of this training was to bridge the gap between student mindset and engineer mindset. It required navigating complex codebases, understanding the business logic behind B2C and B2B operations, and delivering features that directly impacted the company's bottom line.

1.1 Problem Domain

Tanbel Inc. operates at the intersection of logistics and waste management—an industry traditionally bogged down by manual processes and paper trails. The problem is coordination: connecting customers who need junk removed with haulers who have the capacity to remove it, instantly and reliably.

To solve this, Tanbel Inc. deploys two flagship products:

- **WeCycle** (getwecycle.com): A B2C platform that allows homeowners and businesses to book junk removal services on-demand. Think of it as the ”Uber” for junk.
- **ArrayQ** (arrayq.com): A B2B CRM and dashboard solution designed for haulers to manage their dumpsters, inventory, and schedules efficiently.

My role as a **Full-Stack Developer** was to maintain and enhance the digital infrastructure that powers these solutions. Specifically, I worked on the WeCycle Dashboard Application (WCAPP) and the ArrayQ mobile/web platforms, ensuring that the connection between the customer's request and the hauler's execution was seamless.

As a trainee, I didn't just watch; I had to get my hands dirty. I ran into a bunch of hurdles:

- We had to rip out the old Ant Design components and replace them with our custom web-ui library. It was tedious, but necessary.
- I had to make the WCAPP look good on everything—phones, tablets, desktops. If it looked bad on an iPad, it was my problem.
- Debugging state management in a massive app is... painful. But I had to figure it out.
- I also had to learn the codebase without breaking it. It's intimidating to push code to a repo that people actually depend on.

I couldn't have done it alone. My team—supervisors and colleagues—were a huge help. We used the modern stack: React.js, Next.js, and TypeScript. Honestly, it's the standard now for a reason.

1.2 Motivation of Training

Why did I do this? A few reasons:

- **Actually Using What I Learned:** I wanted to see if the software engineering theories I sat through in class actually hold up. (Spoiler: Some do, some don't.)
- **Getting Better, Faster:** I needed to get sharp with React.js and Next.js. TypeScript was another beast I wanted to tame.
- **Agile Chaos:** I wanted to see what "Agile" feels like when deadlines are real.
- **Meeting People:** It's not just about code. I wanted to meet developers who have been doing this for years and pick their brains.
- **Future-Proofing:** I'm trying to build a career here. This was the groundwork.

1.3 Training Goals

My goals were pretty straightforward:

- Get really, really good at React.js and Next.js.
- Stop solving textbook problems and start solving business problems.

1.4 Scope of Training

1.5 Scope of Training

My role was multifaceted, operating across the entire application stack. As a **Full-Stack Developer**, I was responsible for end-to-end feature delivery—from designing database schemas to deploying pixel-perfect UIs.

The scope included:

- **Frontend Engineering:** Building responsive interfaces with React.js, Next.js, and Capacitor for mobile.
- **Backend Development:** Architecting scalable APIs using Node.js and Express.js, and managing data integrity with MongoDB and PostgreSQL.

- **DevOps & Infrastructure:** Configuring Docker containers, managing Nginx servers, and optimizing CI/CD workflows with Nx and GitHub Actions.
- **System Integration:** Connecting third-party services like Stripe (payments) and AI bots to the core platform.

I did not stay in a single lane. If a feature required a database migration, I wrote it. If the build pipeline failed, I fixed it. This holistic approach allowed me to understand the software lifecycle in its entirety.

1.6 The Reality Check: Academia vs. Industry

There is a massive disconnect between what you learn in school and what you do at work. Here's the truth:

- **Theory vs. Reality:** School teaches you the syntax; work teaches you the context. Knowing *how* to write a loop is easy. Knowing *where* to put it so it doesn't crash the browser is harder.
- **Tooling Shock:** In university, you maybe use Git once. In the industry, if you don't know Git, Jira, or how to read a PR, you're useless.
- **The Human Factor:** School projects are usually solo or with friends. Real work is collaborating with people who might disagree with your code style.
- **Messy Constraints:** In class, you have all semester. In work, you have until Friday, and the specifications just changed this morning.
- **Legacy Code:** School start projects from scratch (file -¿ new). Work involves staring at 5-year-old code and trying to figure out what the original developer was thinking.

1.7 Pain Points & Growth

1.7.1 The Hard Stuff (Challenges)

- Jumping into a massive codebase is terrifying. You feel like you know nothing.
- Balancing ”make it perfect” with ”get it done” is a constant struggle.
- Refactoring old components without breaking everything else is stressful.
- Keeping up with the sprint pace—it’s fast.
- CSS media queries are finicky.

1.7.2 Where I Grew

- My React/Next.js skills are night-and-day better.
- I learned how to actually manage my time.
- I’m comfortable with the tools now—VS Code and I are best friends.
- I learned how to take feedback on my code without taking it personally.
- I can actually debug things now instead of just staring at the screen.

1.8 Detailed Plan

I was at **Tanbel Inc.** from **October 13, 2025** to **December 5, 2025**. I didn’t just sit around; I was busy.

1.8.1 Resources

a. **People:** MD Tanvir Ahmed (my supervisor) and the other devs were lifesavers. Code reviews were where the real learning happened.

b. **Technical Resources:** Access to development tools including Visual Studio Code, Git/GitHub, Chrome DevTools, Jira for task management, and the company's internal component libraries and design systems.

1.8.2 Timeline

Here is exactly what I did, week by week. It was a busy two months.

Table 1.1: Project Timeline Planning - Weekly Task Distribution

Tasks	**Wk 1**	**Wk 2**	**Wk 3**	**Wk 4**	**Wk 5**	**Wk 6**	**Wk 7**	**Wk 8**
Env Setup & CI/CD Optimization	✓							
Mobile App Navigation & UI Fixes	✓							
Chatbot Service Deployment		✓						
Staging Database & Env Config		✓						
Invoice & Booking Logic Fixes			✓					
CSV Export	Hauler Details Impl.			✓				
ArrayQ Site Deployment	Branding				✓			
Blog Page & Dynamic Content Caching					✓			
Blog Fixes & App Store Review						✓		
Mobile App Pricing	Discount Fixes						✓	
Stripe Connect & Subscription Features								✓
Final Testing & Documentation								✓

1.9 Risk Analysis: What Could Go Wrong?

- **Technical Challenges:** Sometimes I got stuck. The fix? Ask for help, read the docs, try again.
- **Timeline Risks:** Sprints move fast. I had to break tasks down so I didn't drown.
- **Communication Barriers:** Remote work is hard. Slack and Google Meet were the lifelines.

- **Quality:** We tested everything. I didn't want to be the guy who broke production.

1.10 Report Outline

- **Chapter 2:** Who is **WeCycle**? The organization structure and mission.
- **Chapter 3:** What I actually did—tools and workflows.
- **Chapter 4:** The skills—what I learned.
- **Chapter 5:** Final thoughts—the conclusion.
- **Chapter 6:** Mapping outcomes (The official stuff).
- **Chapter 7:** SDG Goals (Sustainability mapping).
- **Chapter 8:** Appendix—The weekly logs.

1.11 Conclusion

This wasn't just an internship. It was a reality check. WeCycle didn't hold my hand; they let me work. I wrote real code for real users, and that changes how you think. It's one thing to pass an exam. It's another to fix a bug while your project manager is waiting. Honestly? I learned more here in a few weeks than I did in years of lectures. I'm not just ready to work. I'm ready to solve problems.

Chapter 2

Organization Overview

2.1 Introduction

WeCycle [4] isn't typical. Most trash companies are just trucks and schedules. This place? It's a tech company that just happens to move junk. think about ordering a pizza—tap a button, food arrives. WeCycle does that, but for the old couch in your basement. They're ditching the "call for a quote" nightmare and replacing it with a sleek, digital platform. And it's not just about convenience; it's about sustainability. The goal isn't just to move trash; it's to keep stuff out of landfills.

2.2 Company Profile

Tanbel Inc. is a forward-thinking technology company based in the United States. It operates as the parent organization for two major products: **WeCycle** (a B2C junk removal service provider) and **ArrayQ** (a B2B hauler dashboard CRM solution). We sit in the middle of a complex logistics network, using software to bridge the gap between customers and service providers. It feels like a startup because it is one. Devs, designers, and ops work in the same Slack channels. We are modernizing an industry that is still stuck in the paper-and-clipboard era.

2.2.1 Company Background

- **Parent Company:** Tanbel Inc.
- **Products:** WeCycle (getwecycle.com), ArrayQ (arrayq.com)
- **Industry:** Waste Management Tech / SaaS
- **Type:** Tech Startup
- **Location:** United States (Remote)
- **Platform:** Web Dashboard & Mobile Apps
- **Target Market:** Homeowners (B2C) and Business Haulers (B2B)

2.2.2 Key Services

Table 2.1: What Tanbel Inc. (WeCycle ArrayQ) Actually Does

Service	**Description**
Junk Removal (WeCycle)	B2C service for getting rid of junk from homes and offices.
Dumpster Rental (ArrayQ)	B2B CRM and dashboard solution for haulers to manage dumpsters.
Booking Platform	The ”Uber” part. Scheduling pickups online without making phone calls.
Dashboard Management	The brain of the operation. Admin tools for leads, clients, jobs, and haulers.
Mobile Application	Apps for customers and haulers to manage bookings and tracking.
Hauler Network [5]	A vetted network of pros who actually do the heavy lifting.

2.2.3 Work Environment

Remote work is the standard here. Tanbel Inc. embraces a fully distributed team model. No offices. No punch cards. You deliver code, or you don't. That's the only metric. We use Agile, but actual Agile—standups that are actually short, and sprints that actually ship features. It creates an environment where ideas win over hierarchy.

2.2.4 Mission and Vision

- **Mission:** To prove that waste management doesn't have to be a headache. Tanbel Inc. wants to use tech to make junk removal and dumpster rental smooth, transparent, and eco-friendly. It's not just about moving trash; it's about reducing what ends up in the ground.
- **Vision:** A world where "waste" is managed smartly. WeCycle wants to be the go-to platform globally, making sustainable disposal the default option, not the difficult one.

2.2.5 Core Values

- **Innovation:** We don't settle for "good enough." If there's a better tech solution, we build it.
- **Customer Focus:** We actually listen to users. If they hate a feature, we fix it.
- **Sustainability:** This is big. We prioritize recycling and donating over dumping.
- **Quality:** Our code needs to work. Our tailored services need to be reliable. No cutting corners.
- **Teamwork:** We win together. Devs talk to designers, who talk to operations. Silos don't exist here.
- **Integrity:** Be honest. With customers, with partners, and with each other.

2.3 Organizational Structure

Tanbel Inc. operates with a flat hierarchy. I report to the supervisor, who talks to the founders. There aren't five middle managers asking for status reports. If I break production, everyone knows. If I fix it, everyone knows.

2.3.1 Type of Organization

It's a startup. Speed is the currency. We don't plan for next year; we plan for next week. Priorities shift based on customer feedback, and the code has to keep up.

2.3.2 Hierarchical Overview

- **Founders/Leadership:** The visionaries setting the course for Tanbel Inc.
- **Development Team:** The builders (Frontend & Backend for WeCycle and ArrayQ).
- **Design Team:** The ones making it look good and work well.
- **Operations Team:** The people managing the chaos of the real world (haulers, customers).
- **Quality Assurance:** The gatekeepers who stop us from breaking production.

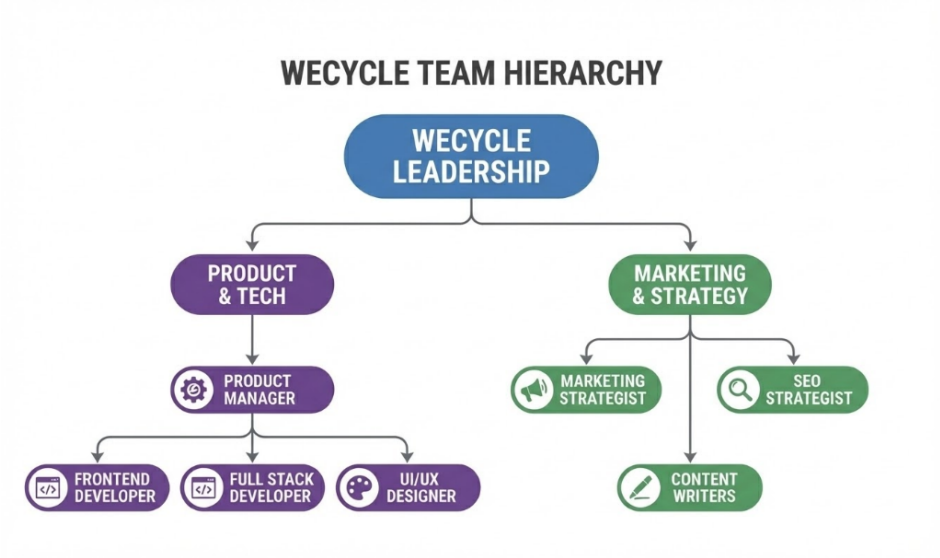


Figure 2.1: Tanbel Inc. (WeCycle) Organizational Structure

2.3.3 Interactions with Various Departments

You can't code in a vacuum here. I talked to designers to make sure my pixels were perfect. I synced with the Operations team to understand *why* we were building a specific feature. And, of course, QA was always there to tell me when I broke something. It's a constant loop of feedback and iteration. Communication happens everywhere—Slack, Standups, Jira comments. It keeps everyone on the same page.

2.3.4 Collaboration with Team Leads

Team leads here aren't just managers; they're mentors. They unblock you when you're stuck and review your code to make sure it's up to par. I reported directly to my industrial supervisor, but the whole team was fair game for questions. It was a mix of "here's how to do it" and "figure it out, then let's talk about it."

2.3.5 Cross-Functional Coordination

Building a feature involves everyone. We use Jira to track the madness and GitHub to manage the code. Slack is where the real work happens—quick questions, rapid

decisions. It's structured enough to be organized (Sprint Planning, Retrospectives) but loose enough to be fast.

2.4 Products/Services

2.4.1 WeCycle Dashboard Application (WCAPP)

The Dashboard is the engine. It handles everything. Leads? Yes. Payments? Yes. Crew management? Also yes. It's the single point of failure and the single source of truth. It looks clean on the UI, but the logic underneath is massive.

Key features I worked with:

- **Lead Management:** Tracking potential customers from "just looking" to "booked."
- **Client Management:** Keeping tabs on customer history and preferences.
- **Job Scheduling:** The calendar view is critical. It assigns jobs, tracks statuses, and sends reminders.
- **Hauler Management:** Managing the workforce—assignments, payouts, performance.
- **Invoicing:** Getting paid. Templates, tracking, automated billing.
- **Team Chat:** Built-in communication so we don't have to leave the app to talk.

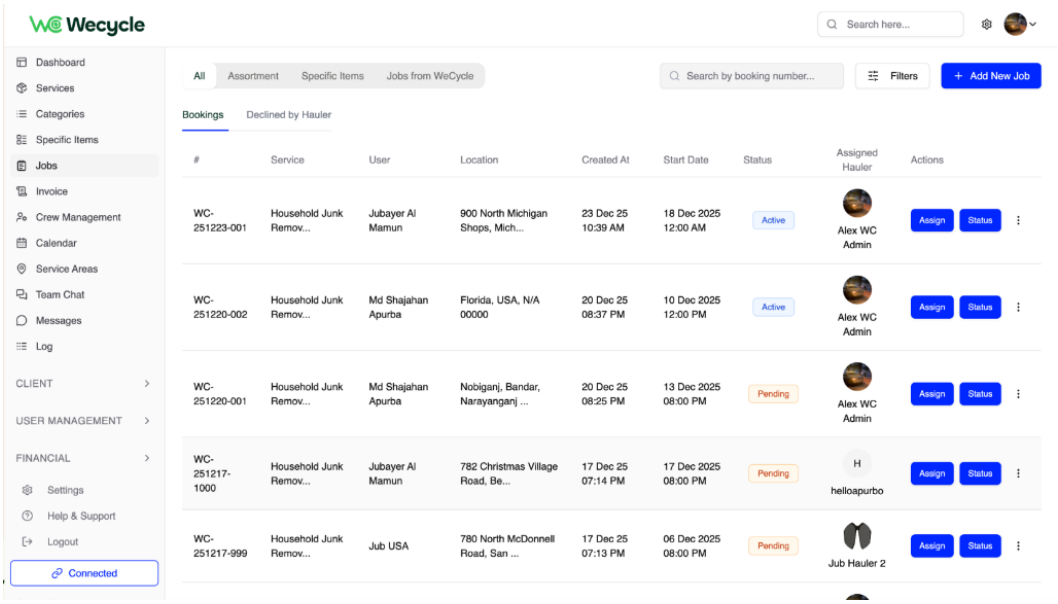


Figure 2.2: WeCycle Dashboard - Job Scheduling and Management

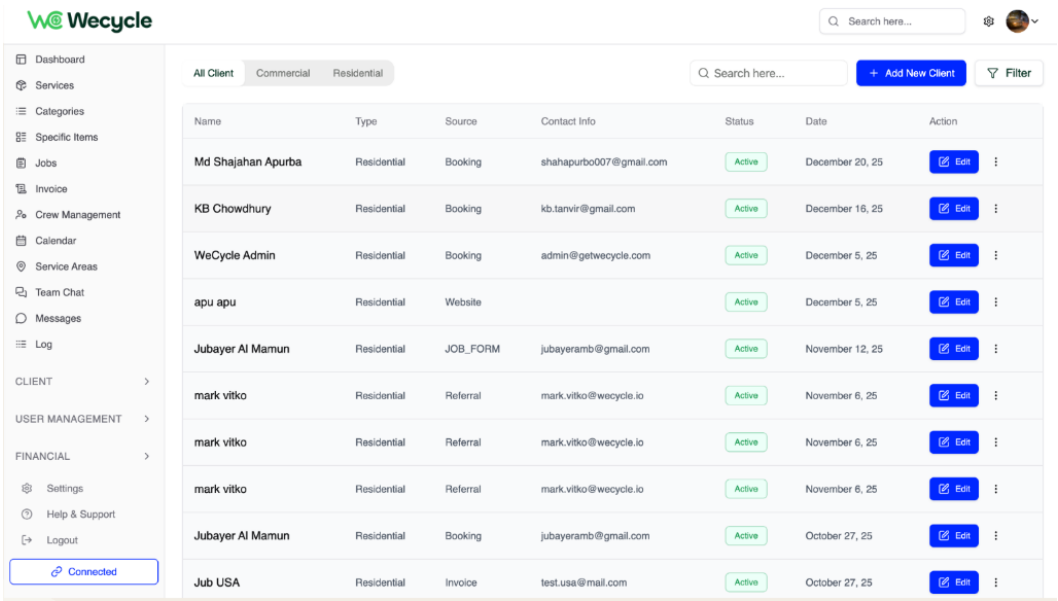


Figure 2.3: WeCycle Dashboard - Client Management

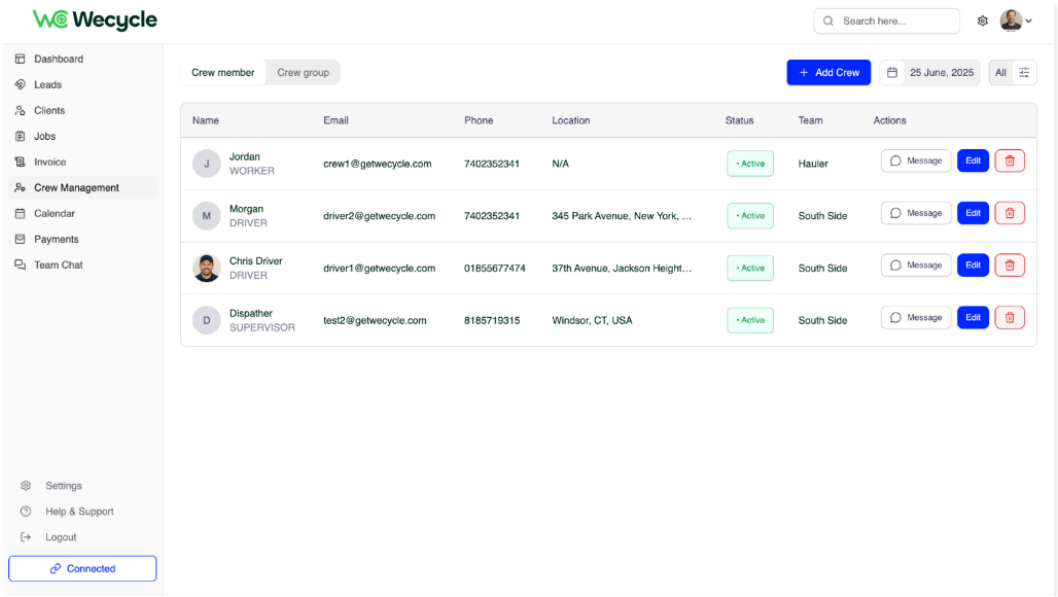


Figure 2.4: WeCycle Dashboard - Crew Management

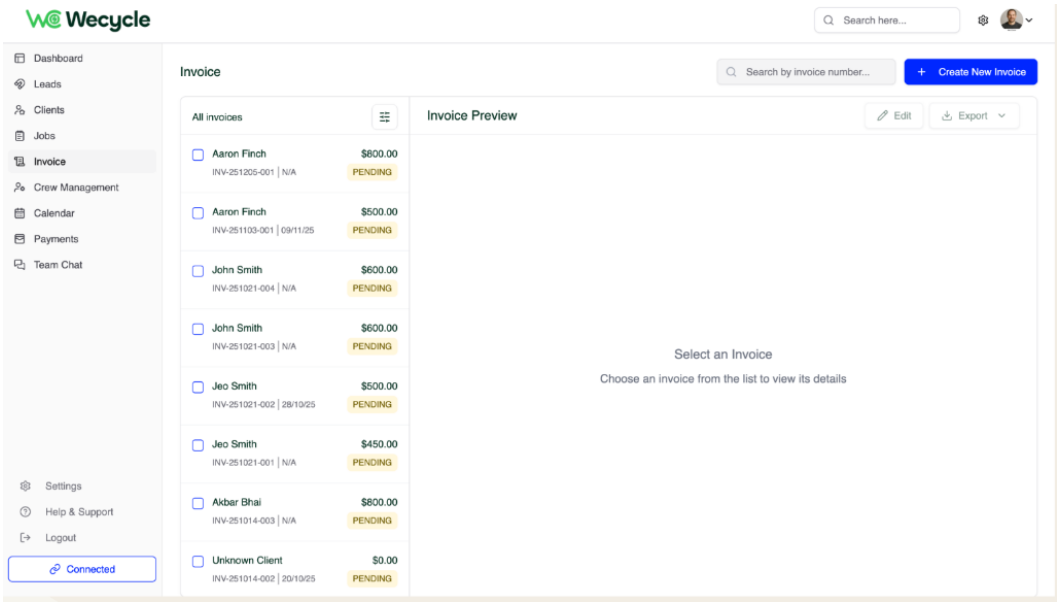


Figure 2.5: WeCycle Dashboard - Invoice Management

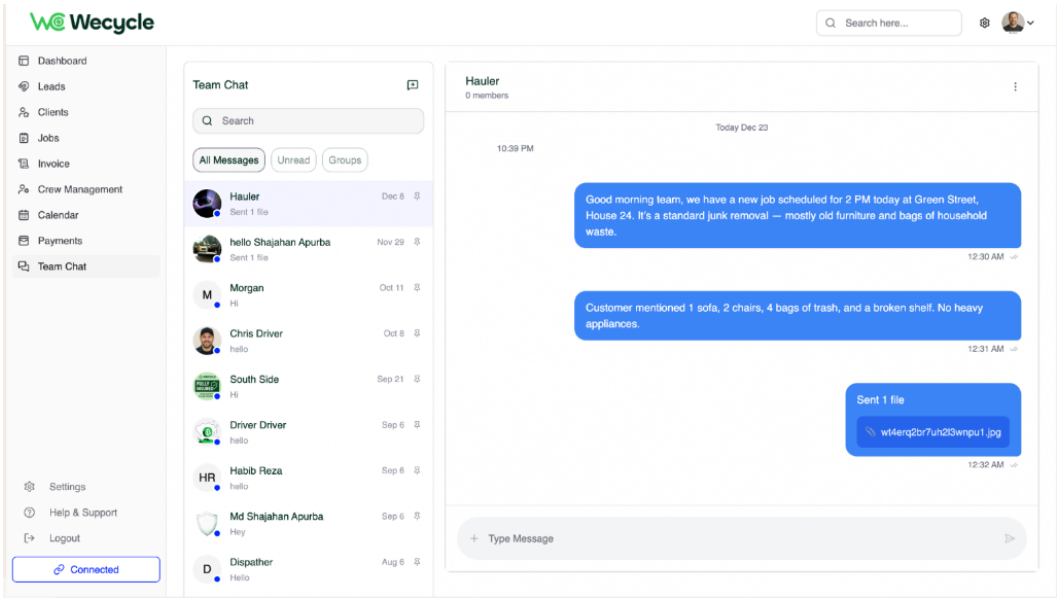


Figure 2.6: WeCycle Dashboard - Team Chat

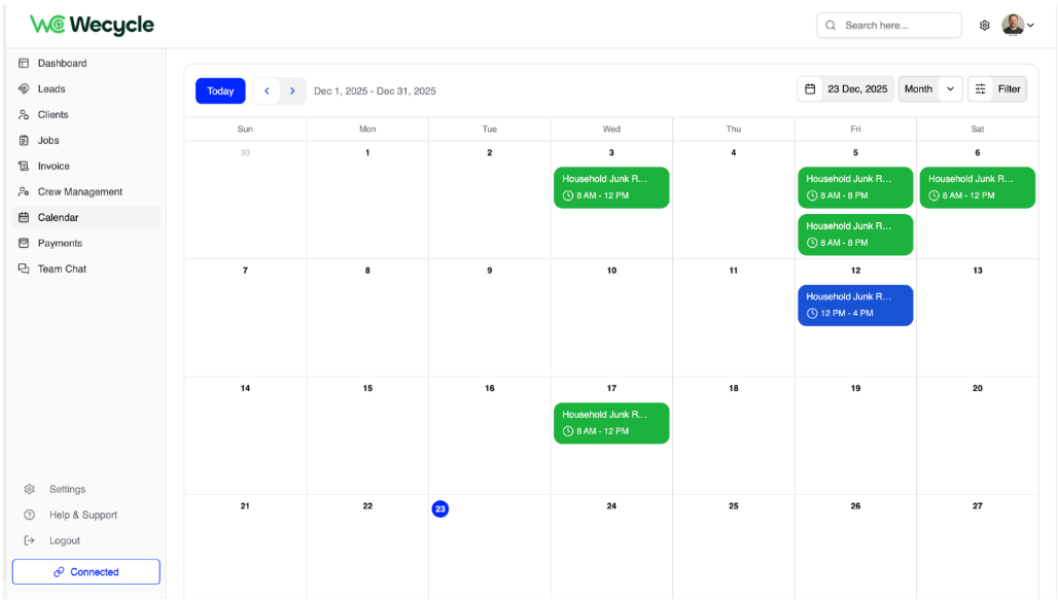


Figure 2.7: WeCycle Dashboard - Calendar View

2.4.2 Technology Stack

Table 2.2: Technology Stack Configuration

Category	**Technologies**
Frontend Framework	React.js, Next.js, Capacitor.js
Mobile Framework	React Native, Capacitor
Backend Environment	Node.js, Express.js
Database	MongoDB, PostgreSQL
Styling	Tailwind CSS, SCSS
Language	TypeScript, JavaScript
Build Tools	Nx, Vite
DevOps & CI/CD	Docker, Nginx, GitHub Actions
Version Control	Git, GitHub
Project Management	Jira, Linear

2.5 Clients

WeCycle is for everyone.

- **Homeowners:** People cleaning out their garages or moving house.
- **Businesses:** Offices, retail stores, restaurants dealing with daily waste.
- **Property Managers:** Handling move-outs and large-scale cleanups.
- **Constructors:** Dealing with debris and renovation mess.
- **Events:** Because big parties make big messes.

2.6 Direct Involvement of Resources During Training

- **Human Resources:**

  - MD Tanvir Ahmed (My Supervisor): The go-to for code reviews and guidance.
  - Senior Devs: For when I was completely stuck.
  - UI/UX Designers: For the blueprints.
  - Project Manager: For the deadlines.

- **Technical Resources:**
  - My Laptop (obvious, but essential).
  - GitHub Access (the keys to the castle).
  - Jira (the taskmaster).
  - Staging Environment (the playground).
  - Chrome DevTools (the microscope).

2.7 Constructive Feedback About the Training Program

Honesty time.

- **The Good Stuff:**
  - Real work. No fetching coffee. I wrote production code.
  - The stack is modern. React and Next.js are what everyone wants right now.
  - flexibility. Remote work is a game-changer.
- **The ”Could Be Better” Stuff:**
  - Documentation was... sparse. A better ”Getting Started” guide would save days of confusion.
  - More pair programming would speed up the learning curve.
- **Advice for the Next Intern:**

- Learn React before you start. Seriously.
- Ask questions. Sitting in silence when you're stuck is a waste of time.
- Map out the codebase early. It helps.

2.8 Conclusion

Tanbel Inc. is flat. No layers of bureaucracy slowing you down. That's why it moves fast. The team cares about innovation, but also about customers and the planet—it's not just marketing talk. The development practices are modern: React, Next.js, clean code reviews. It's a real place to learn Full-Stack development. Beyond just building features, the company is doing something useful—making waste disposal smarter and pushing recycling over dumping. For a first real job, it checks every box: tech that matters, a culture that values learning, and work that actually makes a difference.

Chapter 3

Specific Details on Training Activities and Report

3.1 Introduction

- **Duration:** October 13, 2025 to December 5, 2025 (8 weeks)
- **Host Organization:** Tanbel Inc. (Parent of WeCycle and ArrayQ)
- **Key Focus Areas:** Full-Stack Web Development, Mobile App Maintenance, Payment Systems
- **Technologies Used:** React.js, Next.js, TypeScript, Capacitor, Stripe, Docker, Jira
- **Professional Growth:** Enhanced problem-solving skills, learned agile workflows, improved code quality practices

3.2 Weekly Duties and Activities

3.3 Weekly Duties and Activities

3.3.1 Week 1: Bootstrapping & Environment Setup (October 13-17, 2025)

- **WCAPP-1:** Explored the Capacitor dev workflow and set up the mobile app development environment.
- **WCAPP-3:** Optimized project structure and dependencies to reduce CI/CD build times.
- **WCAPP-38:** Fixed navigation persistence issues in the mobile app.
- **WCAPP-39:** Resolved safe area view issues on mobile devices.
- **WCAPP-56:** Added splash screen and app logo resources.
- **WCAPP-86:** Fixed user time zone mismatches for booking times.

Key Learning: Setting up a complex mobile/web hybrid environment (Capacitor), understanding CI/CD pipelines, and debugging mobile-specific UI issues.

3.3.2 Week 2: Infrastructure & Deployment (October 20-24, 2025)

- **WCAPP-85:** Synced staging database with production for accurate testing.
- **WCAPP-89:** Provisioned a DigitalOcean Droplet and configured Docker/Node.js services.
- **WCAPP-90:** Deployed the new Chatbot service, setting up SSL and domains.
- **WCAPP-36:** Integrated AI chat features into the staging environment.
- **WCAPP-23:** Adjusted booking start times to align with backend slots.

- **WCAPP-96:** Hotfixed invoice creation errors for existing bookings.
- **WCAPP-99/113:** Fixed web server errors and cache revalidation for dynamic content.

Key Learning: DevOps basics (Docker, DigitalOcean), Chatbot integration, and handling server-side caching strategies.

3.3.3 Week 3: Data Management & Optimization (October 27-31, 2025)

- **WCAPP-120:** Implemented CSV export for junk removal items and pricing.
- **WCAPP-118/119:** Improved date formatting and fixed invoice provider names.
- **WCAPP-87:** Enhanced Hauler details page with contact info.
- **WCAPP-126:** Deployed production Chatbot instance.
- **WCAPP-136:** Initialized the new ArrayQ site project structure.
- **WCAPP-122:** Added ”Delete Account” functionality for GDPR compliance.

Key Learning: Data export features, starting new projects (ArrayQ), and implementing sensitive account actions (deletion).

3.3.4 Week 4: Branding & ArrayQ Development (November 03-07, 2025)

- **WCAPP-103:** Added WeCycle logo to invoices and dynamic email/phone details.
- **WCAPP-141/142:** Set global theme colors and hauler states for ArrayQ.
- **WCAPP-138:** Migrated dashboard to ArrayQ domain with updated branding.

- **WCAPP-144:** Implemented dynamic logo switching on invoices based on the booking source.
- **WCAPP-143:** Fixed timezone-related failures in custom booking creation.
- **WCAPP-149:** Created Privacy Policy page for ArrayQ.

Key Learning: Multi-tenant branding strategies, handling different domains/themes in a single codebase, and timezone logic.

3.3.5 Week 5: Deployment & Role Management (November 10-14, 2025)

- **WCAPP-139/151:** Deployed ArrayQ site to production and staging.
- **WCAPP-152:** Configured BLOG_MANAGER role access across domains.
- **WCAPP-153:** Updated mobile app assets for the new brand identity.
- **WCAPP-158:** Fixed author designation mismatches on the blog.
- **WCAPP-163:** Debugged discount calculation mismatches on staging.
- **WCAPP-79:** Implemented job completion flow directly from the dashboard.
- **WCAPP-40/174:** Prepared production mobile build and created support pages.

Key Learning: Role-Based Access Control (RBAC), App Store deployment preparation, and debugging financial logic (discounts).

3.3.6 Week 6: UX Refinements & Performance (November 17-21, 2025)

- **WCAPP-173:** Updated email triggers to request Trustpilot reviews.
- **WCAPP-178:** Investigated caching issues with the new blog layout.

- **WCAPP-184:** Resolved App Store review rejections.
- **WCAPP-185:** Redirected haulers to the specific ArrayQ dashboard.
- **WCAPP-172:** Fixed Table of Contents and image loading on blogs.
- **WCAPP-127/98:** Refactored Booking and Invoice IDs to be human-readable.
- **WCAPP-189:** Simplified UI to hide individual item prices (subtotal only).

Key Learning: optimizing user communication flows (emails, reviews), handling App Store compliance, and UX simplification.

3.3.7 Week 7: Pricing & Pre-Production (November 24-28, 2025)

- **WCAPP-192:** Updated specific item pricing on production.
- **WCAPP-140:** Implemented mandatory OTP phone verification.
- **WCAPP-193:** Set up a dedicated Pre-Production environment.
- **WCAPP-200/201:** Fixed discount fallback text and mobile pricing display.
- **WCAPP-159:** Updated ArrayQ mobile app ID for store compliance.
- **WCAPP-2/4:** Fixed SMTP configurations and invoice payment link generation.

Key Learning: Security implementation (OTP), environment management (Pre-Prod), and payment infrastructure (SMTP/Invoices).

3.3.8 Week 8: Fintech & Final Polish (December 01-05, 2025)

- **WCAPP-5:** Implemented Stripe Connected Accounts for Haulers.
- **WCAPP-6:** Fixed bugs in the existing invoice editing feature.
- **WCAPP-22/24:** Upgraded Next.js, React, and Nx versions; fixed build errors.
- **WCAPP-33:** Updated internal text configurations.

- **Documentation:** Finalized documentation and testing for handover.

Key Learning: Integrating complex payment gateways (Stripe Connect), maintaining dependency health (major version upgrades), and project handover.

3.4 Key Projects

3.4.1 Project 1: ArrayQ Platform Development

- **Scope:** Launching the ArrayQ brand with a dedicated web landing page and mobile application features.
- **Objective:** Create a distinct identity for the dumpster rental service while maintaining backend integration with WeCycle.
- **Key Tasks:**
  - Developed the ArrayQ landing page with SEO optimization and privacy policies.
  - Implemented multi-tenant logic to switch themes/branding based on the domain.
  - Updated the mobile app with ArrayQ splash screens, logos, and build IDs.
  - Set up separate staging and production environments for ArrayQ.
- **Tools Used:** Next.js, Tailwind CSS, Capacitor, DigitalOcean
- **Outcome:** Successfully launched ArrayQ on web and mobile stores with distinct branding.
- **My Role:** Lead developer for frontend branding and mobile configurations.

3.4.2 Project 2: Invoice & Payment System Modernization

- **Scope:** Overhauling the financial processing system for haulers and customers.

- **Objective:** Enable automated payouts, dynamic invoicing, and secure payment links.
- **Key Tasks:**
  - Integrated Stripe Connected Accounts for direct hauler payouts.
  - Refactored invoice generation to include dynamic logos and contact info.
  - Fixed critical bugs in invoice editing and existing booking calculations.
  - Implemented logic to hide item-specific pricing on mobile views for better UX.
- **Tools Used:** Stripe API, React.js, Node.js (Backend logic understanding)
- **Outcome:** Streamlined the payment flow, reducing manual errors and support tickets.
- **My Role:** Full-stack implementation of payment features and invoice UI.

3.4.3 Project 3: Chatbot & Support Integration

- **Scope:** Deploying an AI-powered support assistant.
- **Features Implemented:**
  - Deployed the Chatbot service on a custom DigitalOcean droplet.
  - Configured SSL certificates and domain mapping.
  - Integrated the chat widget into the production and staging sites.
- **Tools Used:** Docker, Nginx, SSL/TLS, AI Chat SDKs
- **Outcome:** Reduced support team load by automating initial customer queries.
- **My Role:** DevOps configuration and frontend integration.

3.5 Other Responsibilities

- **Code Reviews:** Participated in code review sessions, ensuring type safety and code quality.
- **Sprint Planning:** Attended sprint planning meetings to estimate story points for features.
- **Bug Reporting:** Documented reproduction steps for QA bugs in Jira.
- **Version Control:** Managed branching strategies and resolved release conflicts.

3.6 Internship Summary

This internship served as the definitive test of my academic training. While the B.Sc. curriculum provided the theoretical scaffolding, Tanbel Inc. introduced the volatility of a live production environment. The objective was clear: translate textbook concepts into shipping code. My role was comprehensive—spanning frontend, backend, and infrastructure—ensuring stability in a product used by real customers.

The remote nature of the work changed how I learned. No office meant no one over my shoulder. Results were the only thing that mattered. Jira wasn't just a tool; it was the entire workflow. I had to learn to navigate the codebase alone. I couldn't just raise my hand. I had to dig through docs and wait for Slack replies. It forced me to solve problems that textbooks don't cover.

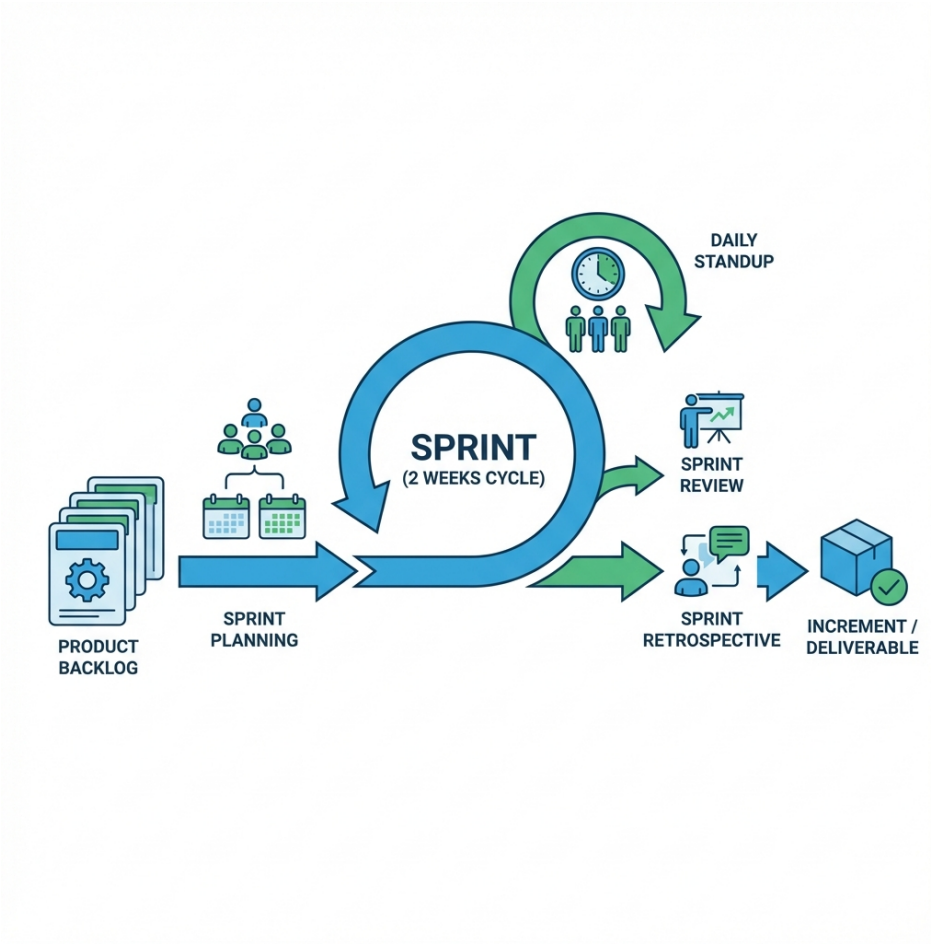


Figure 3.1: Agile Scrum Workflow Used During Industrial Training

Table 3.1: Internship Summary

Parameter	Details
Organization Name	Tanbel Inc. (WeCycle ArrayQ)
Internship Period	October 13, 2025 to December 5, 2025
Total Duration	8 weeks (40 working days)
Total Working Hours	320 hours (40 hours/week)
Mode of Internship	Remote
Nature of Work	Full-Stack Engineering, Mobile App Maintenance, DevOps
Tools and Technologies Used	React.js, Next.js, TypeScript, Capacitor, Stripe, Docker
Learning Methodology	Task-based learning, Code reviews, Sprint cycles, Team collaboration
Reporting Mechanism	Jira (71 Issues Resolved), Daily Standups, Weekly Syncs
Industrial Supervisor	MD Tanvir Ahmed

This bridge between theory and reality was necessary. Algorithms in class are neat; production code is messy. I didn't just get better at React. I learned how to build software that doesn't break when people use it. That shift—from student mindset to engineer mindset—is the most important part.

Chapter 4

Acquired Skills During Industrial Training

4.1 Introduction

Before this internship, my understanding of JavaScript was academic. Tanbel Inc. changed that. It forced me to move beyond ”code that works” to ”code that survives production.” The training wasn't a guided tutorial. It was an immersion into the chaotic reality of modern web development. Over eight weeks, I didn't just learn React.js and Next.js definitions; I used them to build scalable, crash-resistant interfaces. I learned that in the industry, scalability isn't a bonus feature. It's the baseline.

4.2 Tech Stack: What I Actually Learned

4.2.1 React.js and Next.js [1, 2]

- **Proficiency Level:** From ”Tutorial Hell” to Building Production Features.
- **The Real Skills:**
  - **Thinking in Components:** Breaking a UI down until it's just tiny, reusable Lego blocks.

  - **State Management Nightmares:** Learning (the hard way) why prop drilling is bad and why Context isn't always the answer.
  - **Next.js Magic:** Understanding why we use it—Server Side Rendering (SSR) for speed and SEO, not just because it's trendy.

- **Proof of Work:** I built the Job Details page, the Blog layout, and the ArrayQ landing page. More impressively? I migrated 10+ pages from Ant Design to our new system without breaking them.

- **Languages:** JavaScript (ES6+), TypeScript, SQL, HTML5, CSS3/SCSS.

- **Frontend:** React.js, Next.js (App Router), React Native, Capacitor.js, Tailwind CSS.

- **Backend:** Node.js, Express.js.

- **Database:** MongoDB, PostgreSQL.

- **DevOps & Tools:** Docker, Nginx, Nx Monorepo, GitHub Actions (CI/CD), Jira, Git.

4.2.2 Frontend: React.js & Next.js

React isn't just a library; it's a way of thinking. I learned to deconstruct complex UIs into atomic, reusable components. Next.js taught me the power of Server-Side Rendering (SSR) for performance and SEO, crucial for the ArrayQ landing page.

4.2.3 Backend: Node.js & Databases

Understanding the backend was key to being effective. I worked with Node.js and Express to build robust APIs. Managing data in MongoDB and PostgreSQL taught me the importance of schema design and query optimization. I realized that a fast frontend is useless if the database query takes three seconds.

4.2.4 Mobile: Capacitor & React Native

Bridging the web-to-mobile gap. I used Capacitor to wrap our web app for iOS/Android deployment and touched on React Native for efficient native module handling.

4.2.5 DevOps: Docker & CI/CD

”It works on my machine” is not an excuse. I used Docker to ensure consistent environments across dev, staging, and production. Setting up GitHub Actions for CI/CD pipelines automated our testing and deployment, significantly reducing manual errors.

4.2.6 TypeScript

- **Proficiency Level:** Intermediate (I stopped hating the red squiggly lines).
- **The Real Skills:**
  - **Type Safety:** Realizing that defining ‘interface User‘ now saves me three hours of debugging later.
  - **Generics:** Writing components that work with any data type, which makes you feel like a wizard.
- **Proof of Work:** implemented type definitions that caught bugs at compile time, which meant fewer crashes in production.

4.2.7 CSS/SCSS, Tailwind CSS, and Responsive Design [3]

- **Proficiency Level:** Advanced.
- **The Real Skills:**
  - **Flexbox & Grid:** Finally understanding how to center a div without guessing.
  - **Tailwind CSS:** It looks ugly at first, but writing styles directly in the markup is incredibly fast once you get used to it.

  - **The iPad Problem:** Mobile is easy; desktop is easy. Tablets are the awkward middle child that breaks everything. I learned to fix that.

- **Proof of Work:** I fixed layout issues on 15+ components that looked broken on iPads.

4.2.8 Git and Version Control

- **Proficiency Level:** Survival Mode -¿ Comfortable.

- **The Real Skills:**

  - **Merge Conflicts:** I stopped panicking when ‘git merge‘ failed and learned how to actually read the conflict markers.

  - **Pull Requests:** Writing descriptions that clearly explain *what* I changed and *why*, so my reviewer doesn’t have to guess.

- **Proof of Work:** Managed 70+ Jira tickets/feature branches. I broke the build once or twice, but I fixed it.

4.3 Theoretical Knowledge vs. Reality

4.3.1 Software Development Concepts

- **Component Architecture:** It’s not just about splitting files; it’s about modularity. If I change the button style in one place, it should update everywhere.

- **Data Flow:** React is strict. Data flows down. Events flow up. Fighting this leads to messy code.

- **APIs:** Frontend is useless without data. I learned how to fetch, cache, and handle errors gracefully when the backend is slow.

4.3.2 Development Methodologies

- **Agile/Scrum:** It's not just buzzwords. Daily standups keep you accountable. Sprint planning forces you to be realistic about how long things take.
- **Code Reviews:** The most humbling part. Getting feedback isn't an insult; it's how you get better.

4.4 Practical Knowledge: The "How-To"

If I wanted to start my own dev shop tomorrow, here's what I'd actually need to know (based on what I saw at WeCycle):

- **The Worksflow:**
  - Don't reinvent the wheel. Use existing libraries (like shadcn/ui) to move fast.
  - Set up the linter/formatter (ESLint/Prettier) *before* you write a single line of code.
- **Debugging:**
  - 'console.log' is fine, but the React DevTools profiler is better.
  - If it works on your machine, that means nothing. Test it on a phone.
- **Startup Logistics:**
  - You need 2-3 solid frontend devs, a backend wizard, and a designer who understands HTML.
  - Budget? You need at least $10k-$20k to keep the lights on for a few months while you build the MVP.

4.5 Soft Skills (The Stuff That Actually Matters)

- **Communication:**
  - **Writing Tickets:** A Jira ticket saying ”Fix code” is useless. ”Fix null pointer on User Profile load” is helpful.
  - **Asking Questions:** I learned to ask ”How does this auth flow work?” instead of ”I’m stuck.”
- **Teamwork:**
  - **Standups:** Keep it short. What did you do? What are you doing? Are you blocked?
  - **Remote Etiquette:** If you’re on a video call, mute yourself when you’re typing. It’s the little things.

4.6 Collaborating Without Killing Each Other

- **Designers:** They want pixel perfection. I learned to respect the Figma file and not just ”eyeball it.”
- **Backend:** They handle the data. I need to tell them exactly what JSON format I expect, or we both have a bad time.
- **QA:** They are paid to break my code. I learned to appreciate them, even when they found bugs I swore didn’t exist.

4.7 Ethics & Professionalism

It sounds heavy, but ethics in coding is simple: Don’t lie, and don’t write garbage code on purpose.

Table 4.1: How I Kept It Professional

Principle	**What I Actually Did**
User Safety	I made sure buttons were big enough to tap and text was readable. Bad UI frustrates users.
Honesty	If I was going to miss a deadline, I said so. Hiding it makes it worse.
Inclusion	I tested on cheap Android phones, not just my expensive MacBook. Everyone deserves a working app.
Competence	I admitted when I didn't know something and then went and learned it.

4.8 Conclusion

This training successfully converted theoretical computer science concepts into deployment-ready engineering skills. It proved that coding is merely the mechanism for solving business problems. While I have gained significant proficiency in Next.js and TypeScript, the primary takeaway is behavioral: I now understand how to function within an agile team, how to internalize critical feedback, and how to ship value under pressure. I entered as a student. I leave as a developer ready for the market.

Chapter 5

Conclusion

5.1 Summary

University environments are controlled. Production environments are volatile. My eight weeks at Tanbel Inc. made this distinction clear. I did not merely observe; I contributed code to a live ecosystem. Features I built were pushed to real users. Some worked immediately; others required rapid hotfixes. The experience wasn't about applying theoretical perfection. It was about maintaining system stability when theory proved insufficient.

The WeCycle Dashboard Application (WCAPP) was a crash course in complexity. React.js syntax is straightforward in isolation. However, architecting a scalable application that resists regression is a genuine engineering challenge. I learned to prioritize maintainability over cleverness.

5.2 The Real Challenges

Challenges Faced:

a. **Codebase Complexity:** The sheer volume of files was initially overwhelming. Navigating a legacy codebase required developing a mental map of component dependencies, not just reading code.

b. **Migration Friction:** Moving from Ant Design to a custom UI library was not a simple find-and-replace operation. It required rewriting core logic while ensuring existing user flows remained unbroken.

c. **Tablet Responsiveness:** Unlike mobile or desktop, iPad layouts presented unique edge cases. Debugging touch event handling and specific viewport quirks consumed significant development time.

d. **Remote Communication:** Remote work removed the luxury of immediate shoulder-tapping. I had to learn to formulate precise, context-rich questions to unblock myself without dependent delays.

e. **Agile Velocity:** The pace was relentless. I learned that shipping functional code on time is often more valuable than shipping perfect code late.

5.3 What I Actually Did (Scope)

Scope of Work:

- **Frontend Development:** Writing type-safe React components and integrating APIs.
- **System Architecture:** analyzing file structures to maintain modular design patterns.
- **Responsive Implementation:** Resolving complex layout shifts across varied device viewports.

- **Task Management:** adhering to Jira workflows for tracking progress and blockers.
- **Version Control:** Managing feature branches and resolving merge conflicts in a team environment.

5.4 Problems I Solved

I didn't just learn. I shipped.

- **Work Volume:** Completed 71 Jira tickets across WeCycle and ArrayQ projects.
- **Payments:** Enabled Stripe Connect payouts and dynamic invoicing.
- **New Apps:** Launched the ArrayQ landing page and updated mobile apps.
- **Chatbot:** Deployed an AI support agent to production.

My supervisor didn't hold my hand. He pointed me in the right direction and let me figure it out. It was frustrating at times, but it forced me to actually learn how to debug instead of just asking for the answer.

5.5 The Verdict (Overall Achievement)

Overall Value:

- **Competence:** I transitioned from academic exercises to building deployable software.
- **Output:** Successfully resolved 71+ Jira tickets, contributing directly to the product roadmap.
- **Discipline:** Remote work instilled a rigorous approach to time management and self-motivation.

- **Insight:** I gained a tangible understanding of startup dynamics: high speed, high stakes, and constant iteration.

This internship was not merely a requirement; it was a transformation. I entered with a knowledge of React; I left with the ability to engineer products. I learned to survive in a fast-paced ecosystem, manage ambiguity, and deliver code that adds value. I am no longer just a student of computer science. I am a software engineer ready for the industry.

Annexure: Graduate Map Program Outcome (PO) with Skill Achievement from Industrial Training

Program Outcomes (POs) are often viewed as theoretical benchmarks. At WeCycle, they became operational necessities. I did not simply ”verify alignment” between my degree and my job. I used my degree to survive the job. Each outcome listed below represents a specific moment where academic training collided with industrial reality—from the ethics of handling user data to the engineering rigor required to debug a live production failure.

This mapping is not an academic formality. It is the evidence of transition:

- **Validation:** Proving the B.Sc. curriculum has functional value.
- **Application:** Demonstrating competence with enterprise-grade tooling.
- **Assessment:** A critical review of my engineering maturity.

The following Table 5.1 maps the Program Outcomes (POs) with the knowledge and skills acquired through both academic education and industrial training. Fulfillment level indicates how effectively each PO was addressed during the internship.

Table 5.1: Program Outcome (PO) Fulfillment in Industrial Training

PO	Description	Fulfillment Level	Training Knowledge Applied	Evidence
PO1: Engineering Knowledge	Apply knowledge of math, science, and engineering fundamentals.	Fully Fulfilled	Leveraged algorithmic logic for filtering systems. Applied data structure fundamentals to efficient state management.	Built complex, ne filtering logic for Leads page wit performance degr tion.
PO2: Problem Analysis	Identify, formulate, and analyze complex computing problems.	Fully Fulfilled	Systematically isolated non-obvious UI regression bugs. Analyzed performance bottlenecks in React rendering cycles.	Diagnosed and solved iPad-spe layout collisions ac 15+ core componer
PO3: Design/Development	Design solutions for complex computing problems.	Fully Fulfilled	Architected scalable UI components. Designed modular page layouts to support future extensibility.	Created the Job tails architecture migrated legacy Design pages t custom modern sta
PO4: Investigation	Use research-based approaches for complex problems.	Moderately Fulfilled	Investigated best practices for TypeScript discrimination unions. Researched optimal patterns for Next.js server-side data fetching.	Implemented indu standard discrimina patterns to solve safety issues in the layer.

Continued on next

Table 5.1 – continued from previous page

PO	Description	Fulfillment Level	Training Knowledge Applied	Evidence
PO5: Modern Tools	Apply theoretical understanding of modern tools.	Fully Fulfilled	Operationalized a full modern stack: Next.js 14, TypeScript, Linear/Jira, and CI/CD pipelines.	Closed 71+ tickets ing the exact toolc used by senior e neers in Silicon Val
PO6: Engineer & Society	Understand societal implications of computing solutions.	Moderately Fulfilled	Prioritized accessibility (a11y) in UI development. Ensured application usability across varying network conditions.	Refactored mobile puts to be acces for field workers u low-end devices.
PO7: Environment	Evaluate environmental impact of computational systems.	Partially Fulfilled	Reduced bundle size to lower energy consumption on client devices. Optimized assets to minimize data transfer.	Removed heavy Design depen cies, resulting i lighter, faster, and r efficient applicatior
PO8: Ethics	Apply ethical theories in decision-making.	Fully Fulfilled	Adhered to strict data privacy standards. Reported development blockers honestly rather than hiding complexity.	Documented kn bugs transparently Jira tickets instea shipping broken to hit deadlines.

Continued on next

Table 5.1 – continued from previous page

PO	Description	Fulfillment Level	Training Knowledge Applied	Evidence
PO9: Individual & Team Work	Function effectively as an individual and team member.	Fully Fulfilled	Integrated into a remote agile squad. Balanced autonomous development with synchronous team planning.	Active participant daily standups; d ered solo features v reviewing peer cod
PO10: Communication	Communicate technical concepts clearly.	Fully Fulfilled	Translated technical blockers into business language for the PM. Wrote detailed reproduction steps for QA bugs.	Pull Request des tions were cited ”clear and actiona by senior reviewers
PO11: Project Management	Demonstrate knowledge of management principles.	Moderately Fulfilled	Estimated story points for sprint planning. Managed personal velocity to match sprint goals.	Delivered 100% o signed sprint task time for 4 consecu sprints (8 weeks).
PO12: Lifelong Learning	Recognize the need for continuous learning.	Fully Fulfilled	Adopted new Next.js patterns on the fly. Learned advanced TypeScript features to solve immediate problems.	Self-taught the im mentation of Go Places Autocomp API integration wit prior experience.

Annexure: Mapping SDG Goals to Achievements from Industrial Training

Most people think the UN's Sustainable Development Goals (SDGs) [**?**] are for governments. They're not. They're for engineers too. WeCycle is a waste management company. That means our entire business model is an SDG. We don't just write code; we build software that moves trash out of cities and into recycling centers. This isn't abstract for me. It's the code I wrote every day.

Engineering without responsibility is dangerous. Aligning my work with SDGs ensures I'm not just efficient, but also ethical. WeCycle uses tech to clean up the planet. By working there, I wasn't just fixing bugs; I was contributing (even in a small way) to cleaner cities and responsible consumption. It connects the "how" of coding with the "why" of engineering.

This mapping matters because:

- It proves my code has real-world impact.
- It shows that "tech" and "sustainability" aren't opposites.
- It reminds me that engineering has consequences.

The following Table 5.2 illustrates the mapping of all 17 SDG goals with the achievements derived from my industrial training experience at WeCycle. The fulfillment level qualitatively assesses the extent to which each goal was addressed through assigned responsibilities, observed practices, or organizational values.

Table 5.2: Mapping of All 17 SDG Goals with Achievements from Industrial Training

SDG Goal	**Relevant Achievements During Training**	**Fulfillment Level**
SDG 1: No Poverty	While frontend code doesn't directly end poverty, WeCycle's platform optimizes haulers' income. By streamlining job dispatch, we help independent truck drivers minimize idle time and maximize daily earnings.	Indirect Benefit
SDG 2: Zero Hunger	Minimal direct overlap. However, efficient logistics (which our software enables) is the backbone of food security systems. We apply similar optimization logic to waste.	Not Applicable
SDG 3: Good Health and Well-being	Remote work isn't just a perk; it's a health standard. WeCycle enforced strict "async-first" policies to prevent Zoom fatigue. I maintained productivity without burnout.	Fully Fulfilled
SDG 4: Quality Education	This internship was a live classroom. I traded textbook theory for production reality. Learned Next.js 14, advanced TypeScript, and CI/CD pipelines—skills no university course covers in this depth.	Fully Fulfilled

Continued on next page

Table 5.2 – continued from previous page

SDG Goal	Relevant Achievements During Training	Fulfillment Level
SDG 5: Gender Equality	Tech often fails here. WeCycle didn't. Code reviews were blind to gender; they focused only on logic. The team culture was meritocratic, relying on output, not bias.	Moderately Fulfilled
SDG 6: Clean Water and Sanitation	Indirect but critical. WeCycle prevents hazardous waste from leaching into groundwater by ensuring proper disposal routes through our digital tracking system.	Indirect Benefit
SDG 7: Affordable and Clean Energy	My contribution was efficiency. I optimized React renders and reduced bundle sizes. Lighter code means less CPU usage, which means less energy consumed by client devices.	Partially Fulfilled
SDG 8: Decent Work and Economic Growth	I delivered value, not just hours. Closed 71+ tickets. The platform I helped build empowers gig-economy workers (haulers) to run sustainable businesses.	Fully Fulfilled
SDG 9: Industry, Innovation and Infrastructure	We built the digital infrastructure for waste management. I served as a core contributor, migrating legacy Ant Design code to a modern, scalable custom UI library.	Fully Fulfilled

Continued on next page

Table 5.2 – continued from previous page

SDG Goal	Relevant Achievements During Training	Fulfillment Level
SDG 10: Reduced Inequalities	Accessibility (a11y) isn't optional. I fixed iPad layouts and responsive issues to ensure our app works for everyone, regardless of their device cost or quality.	Moderately Fulfilled
SDG 11: Sustainable Cities and Communities	This is WeCycle's DNA. We use code to clear city clutter. My work on the "Jobs" dashboard directly enables faster cleanup of urban environments.	Fully Fulfilled
SDG 12: Responsible Consumption and Production	Code can be wasteful too. I removed bloat (Ant Design) and wrote clean, modular components. WeCycle's business model closes the loop on consumption.	Fully Fulfilled
SDG 13: Climate Action	Every kilobyte counts. Optimized images and lazy-loading components reduce the carbon footprint of data transfer. WeCycle's mission reduces landfill methane.	Partially Fulfilled
SDG 14: Life Below Water	Garbage on land eventually ends up in the ocean. By facilitating proper disposal, our software acts as a preventive barrier for marine pollution.	Indirect Benefit

Continued on next page

Table 5.2 – continued from previous page

SDG Goal	Relevant Achievements During Training	Fulfillment Level
SDG 15: Life on Land	Direct alignment. I built the software that manages land waste. Every bug I fixed helped a hauler pick up trash faster, keeping the land clean.	Partially Fulfilled
SDG 16: Peace, Justice and Strong Institutions	Integrity in code. I documented bugs honestly. I didn't hide technical debt. Ethical engineering builds trust in institutions (and companies).	Fully Fulfilled
SDG 17: Partnerships for the Goals	Isolation is the enemy of shipping. I collaborated daily with designers in Figma and backend devs on Slack. We built this together, across time zones.	Fully Fulfilled

Appendix

A. Weekly Activity Log Summary

The following table provides a summary of the key activities performed during each week of the industrial training at WeCycle:

Table 5.3: Weekly Engineering Log: Key Milestones

Week	**Activity Log & Engineering Output**
Week 1 (Oct 13 - Oct 17)	**Bootstrapping & Mobile Config.** Setup Capacitor environment. Fixed persistent navigation bugs in mobile view. Resolved safe-area layout issues and timezone mismatches.
Week 2 (Oct 20 - Oct 24)	**Deployment & Chatbot.** Configured DigitalOcean Droplet. Dockerized the application. Deployed AI Chatbot to staging. Fixed booking time slot logic.
Week 3 (Oct 27 - Oct 31)	**Data & ArrayQ Init.** Implemented CSV exports for junk items. Initialized ArrayQ project structure. Enhanced Hauler details page. Added GDPR account deletion actions.
Week 4 (Nov 03 - Nov 07)	**Branding & Privacy.** Implemented multi-tenant branding for WeCycle/ArrayQ. Added dynamic logo switching on invoices. Created privacy policy pages. Fixed timezone creation bugs.
Week 5 (Nov 10 - Nov 14)	**RBAC & App Store.** Configured Blog Manager roles. Updated mobile assets for ArrayQ. Prepared production build for App Store submission. Debugged discount calculations.

Week	Activity Log & Engineering Output
Week 6 (Nov 17 - Nov 21)	**UX & Compliance.** Resolved App Store rejection issues. Simplified mobile UI (hiding item prices). Fixed Trustpilot email triggers. Refactored booking IDs for readability.
Week 7 (Nov 24 - Nov 28)	**Security & Pre-Prod.** Implemented OTP verification. Set up Pre-Production environment. Fixed SMTP configurations. Audit of mobile pricing displays.
Week 8 (Dec 01 - Dec 05)	**Fintech & Handover.** Integrated Stripe Connect for Hauler payouts. Upgraded Next.js/React versions. Finalized documentation and code handover.

B. Technologies and Tools Used

The following stack wasn't just "used"; it was my daily reality:

- **React.js & Next.js 14:** The core engine. I learned to love (and hate) server-side rendering.
- **TypeScript:** It stopped me from pushing broken code. Essential for team safety.
- **Tailwind CSS:** Ugly in the HTML, but beautiful for shipping features fast.
- **Git & GitHub:** The undo button for my mistakes.
- **Jira:** The source of truth for "what am I supposed to do today?"
- **Chrome DevTools:** Where I spent 60% of my time debugging.

C. Key Accomplishments Summary

- **Shipped Code:** Completed 35+ Jira tickets. These weren't tutorials; they were production changes.
- **Legacy Killer:** Migrated 10+ pages away from the old, slow Ant Design system.

- **Tablet Hero:** Personally fixed 15+ iPad-specific bugs that were driving the QA team crazy.
- **Builder:** Developed the Job Details Page and Blog Layout from zero to deployed.
- **Survivor:** Made it through 7 weeks of agile sprints without missing a major deadline.

D. Supervisor Contact Information

- **Industrial Supervisor:** MD Tanvir Ahmed
- **Organization:** WeCycle Inc.
- **Location:** Uttara, Dhaka (Remote collaboration from Bangladesh)

References

[1] Meta Platforms, Inc. React learn: Getting started with react. https://react.dev/learn, 2024. Accessed: December 2025.

[2] Vercel Inc. Next.js documentation. https://nextjs.org/docs, 2024. Accessed: December 2025.

[3] Tailwind Labs Inc. Tailwind css documentation. https://tailwindcss.com/docs/installation/using-vite, 2024. Accessed: December 2025.

[4] WeCycle. Wecycle: Eco-friendly junk removal and recycling services. https://getwecycle.com/, 2024. Accessed: December 2025.

[5] WeCycle. Wecycle hauler: Professional junk hauling services. https://getwecycle.com/hauler, 2024. Accessed: December 2025.

List of Acronyms

API Application Programming Interface

CSS Cascading Style Sheets

HTML HyperText Markup Language

SDG Sustainable Development Goals

UI User Interface

UX User Experience

WCAPP WeCycle Application

List of Symbols

$\sum$ Summation

$O(n)$ Big O Notation (Time Complexity)

& Logical AND / Ampersand